

WEBTECH NETWORK • WORKSHOP

Vue.js e Nuxt.js

Do primeiro componente a uma aplicação completa,
com rotas, consumo de API e SEO.

Artur Bomtempo





SOBRE MIM

Artur Bomtempo

Chapter Lead no WebTech Network · Desenvolvedor de Software

Graduando em Engenharia de Software na PUC Minas, com mais de 4 anos de experiência em desenvolvimento web moderno e paixão por ensinar.

Chapter Lead

Lidero o capítulo do WebTech Network, conectando pessoas a tecnologia na prática.

Full stack

Desenvolvimento web moderno com React, Node.js, Java e Spring Boot.

Educação

Materiais de apoio para alunos e conteúdos de programação no YouTube.

arturbomtempo.dev

[GitHub: arturbomtempo-dev](https://github.com/arturbomtempo-dev)

O que vamos ver hoje

Parte 1 · Vue.js

- 01 História e conceitos**
Como o Vue surgiu e o que ele resolve
- 02 Anatomia de um .vue**
script, template, style, setup e scoped
- 03 Reatividade e diretivas**
ref, computed, v-bind, v-on, v-if, v-for
- 04 Componentes**
Props, slots, emits e ciclo de vida

Parte 2 · Nuxt.js

- 05 História e vantagens**
Por que usar um meta-framework
- 06 Criando o projeto**
Setup, Prettier e estrutura
- 07 Rotas, layout e API**
pages/, layouts/ e useFetch
- 08 Refinamentos e desafio**
Erro 404, useHead, filtros e busca

01

Vue.js

O framework progressivo para construir interfaces reativas a partir de dados.

NESTA PARTE

- 01 História e conceitos
- 02 Anatomia de um .vue
- 03 Reatividade
- 04 Diretivas
- 05 Componentes

Como o Vue surgiu

Evan You trabalhava no Google com AngularJS e queria algo mais leve: ficar só com a parte boa, a ligação entre dados e tela, sem o peso de um framework enorme.

2014

Lançamento

Vue.js é publicado como projeto open source focado na camada de visão.

2016

Vue 2

Virtual DOM, mais performance e explosão de adoção na comunidade.

2020

Vue 3

Reescrito em TypeScript, com a Composition API e núcleo menor.

2021

script setup

A sintaxe enxuta que vamos usar hoje se torna estável.

Hoje

Vue 3.6

Versão usada no workshop. Vue 2 teve fim de suporte em 2023.

O que é o Vue



Declarativo

Você descreve como a tela deve ficar a partir dos dados, e o Vue cuida de atualizar o HTML.



Reativo

Quando um dado muda, a tela muda junto, sem `querySelector` nem `innerHTML`.



Baseado em componentes

A interface vira peças reutilizáveis (header, card, botão), cada uma com HTML, lógica e CSS.



Progressivo

Dá para usar num pedaço de uma página ou num projeto inteiro.



O Nuxt é construído em cima do Vue. Tudo desta parte vale lá também: o Nuxt só acrescenta rotas, servidor e importação automática.

Do JS puro para o Vue

Antes: montando HTML na mão

```
script.js
const card = `
  <article class="card">
    <h3>${character.name}</h3>
    <p>${character.species}</p>
  </article>`;

list.innerHTML += card;
```

Você controla o DOM manualmente e precisa lembrar de atualizar tudo.

Com Vue: um componente reutilizável

```
App.vue
<CharacterCard
  v-for="character in characters"
  :key="character.id"
  :name="character.name"
  :species="character.species"
/>
```

Você descreve o resultado. Mudou o array, a lista se atualiza sozinha.

Criando um projeto Vue

Terminal

```
npm init vue@latest vue-basics
```

Pergunta	Resposta
Usar TypeScript?	Sim
Recursos	Router
Versão	Vue 3.6 (RC)
Projeto em branco?	Sim

Rodar a aplicação

```
npm run dev
```

Estrutura base

public/ Arquivos estáticos, servidos como estão

src/ O código da aplicação

src/main.ts Ponto de entrada: cria e monta o app

index.html Página base onde o app é montado

vite.config.ts Configuração do Vite, o bundler

package.json Dependências e scripts

A estrutura de um arquivo .vue

```
Title.vue

<script setup>
const title = 'Olá, Elmore!';
</script>

<template>
  <h1 class="title">{{ title }}</h1>
</template>

<style scoped>
.title {
  color: #3db8e0;
}
</style>
```

O arquivo .vue é um **SFC** (Single File Component), com três blocos:

<script>

A lógica: variáveis, funções, dados.

<template>

O HTML da tela, que usa o que foi criado no script.

<style>

O CSS do componente.

Nenhum bloco é obrigatório, mas quase sempre existe pelo menos o template.

setup e lang

Sem setup (Options API)

```
<script>
export default {
  data() {
    return { title: 'Olá!' };
  },
};
</script>
```

```
<script setup lang="ts">
const age: number = 12;
</script>
```

Com setup (forma moderna e recomendada)

```
<script setup>
const title = 'Olá!';
</script>
```

Tudo que você cria no script (variáveis, funções, imports) fica disponível direto no template.

lang indica a linguagem do bloco. Sem ele, o script é JavaScript. Vale para CSS também: **<style lang="scss">**.

style e scoped

Sem scoped

```
<style>
p {
  color: red;
}
</style>
```



CSS global

Pinta de vermelho **todos** os parágrafos do site.

Com scoped

```
<style scoped>
p {
  color: red;
}
</style>
```



CSS do componente

Vale **só** para este componente. Nada vaza para o resto da página.



Por baixo, o Vue coloca um atributo único no elemento (**data-v-1a2b3c**) e no seletor (**p[data-v-1a2b3c]**). No projeto: design system global em assets/css/ e cada componente com seu style scoped.

Variáveis e interpolação {{ }}

```
<script setup>
const name = 'Gumball';
const age = 12;
</script>

<template>
  <p>{{ name }} tem {{ age }} anos.</p>
</template>
```

O que está no script aparece no template com chaves duplas.

Qualquer expressão JavaScript

```
<p>{{ firstName + ' ' + lastName }}</p>
<p>{{ `${firstName} ${lastName}` }}</p>
<p>{{ firstName.toUpperCase() }}</p>
<p>{{ age + 5 }} anos daqui a 5 anos</p>
<p>{{ age ≥ 18 ? 'Adulto' : 'Criança' }}</p>
```



Dentro de `{{ }}` só cabem **expressões**. Comandos como `if` e `for` não funcionam: para isso existem as diretivas `v-if` e `v-for`.

Reatividade com ref

Counter.vue

```
<script setup>
import { ref } from 'vue';

const count = ref(0);

function increment() {
  count.value++;
}
</script>

<template>
  <p>Cliques: {{ count }}</p>
  <button @click="increment">Clicar</button>
</template>
```

Uma variável comum **não** atualiza a tela quando muda. O ref cria um valor reativo.



No script, use .value

Para ler ou alterar: `count.value++`



No template, sem .value

O Vue desembrulha sozinho: `{{ count }}`



No Nuxt é automático

ref, computed e outros são importados sem **import**.

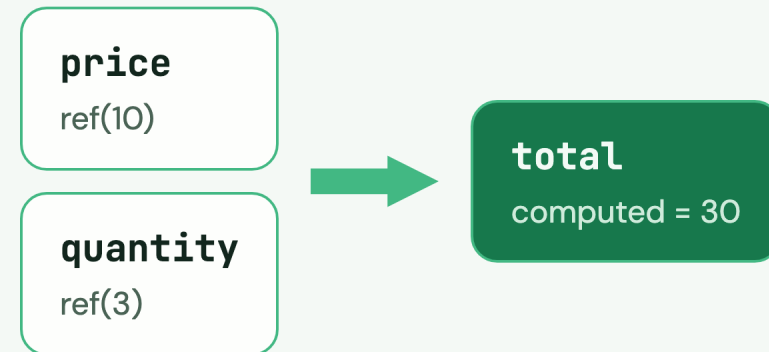
computed: valores derivados

```
<script setup>
import { computed, ref } from 'vue';

const price = ref(10);
const quantity = ref(3);

const total = computed(
  () => price.value * quantity.value
);
</script>

<template>
  <p>Total: R$ {{ total }}</p>
</template>
```



O computed **recalcula sozinho** quando algum valor usado dentro dele muda. Mudou price ou quantity, o total acompanha.

Use para qualquer valor que depende de outros: totais, filtros, textos formatados.

v-bind e o atalho :

```
<script setup>
const image = 'https://exemplo.com/gumball.png';
const name = 'Gumball';
</script>

<template>
  
  
</template>
```

As duas linhas são iguais: **:** é o atalho de **v-bind**.

{{ }} funciona no **texto**. Para colocar uma variável num **atributo**, use v-bind.



Não funciona

O Vue não interpreta chaves em atributo: **src="{{ image }}"**



Entre aspas é JavaScript

Com **:**, dá até para montar textos:
:href="/characters/\${id}"

Classes e estilos dinâmicos

```
<script setup>
import { ref } from 'vue';

const isActive = ref(true);
const color = '#ff7a1a';
</script>

<template>
  <button
    class="chip"
    :class="{ 'is-active': isActive }"
  >Todos</button>

  <div :style="{ backgroundColor: color }">
    Laranja
  </div>
</template>
```



:class com objeto

{ 'is-active': isActive } significa: aplica a classe is-active se isActive for verdadeiro.



class e :class juntos

A classe fixa (chip) continua, e a dinâmica é somada a ela.



:style com objeto

As propriedades CSS são escritas em camelCase: backgroundColor.

v-on e o atalho @: eventos

```
<script setup>
function sayHello() {
  alert('Olá!');
}
</script>

<template>
  <button v-on:click="sayHello">Clique</button>
  <button @click="sayHello">Clique</button>
</template>
```

@ é o atalho de **v-on**. Funciona com qualquer evento:

@click

@input

@submit

@mouseenter

Também dá para escrever a ação direto ou
passar argumentos:

```
<button @click="count++">+1</button>
<button @click="select(2)">Darwin</button>
```

v-if, v-else-if e v-else

```
<script setup>
const status = 'alive';
</script>

<template>
  <p v-if="status === 'alive'">Vivo</p>
  <p v-else-if="status === 'deceased'">Morto</p>
  <p v-else>Desconhecido</p>
</template>
```



Remove do HTML

Se a condição for falsa, o elemento **não existe** no HTML.



Ordem importa

v-else-if e v-else precisam vir **logo depois** do elemento com v-if.



Bônus: v-show

O elemento existe, mas fica escondido com display: none. Bom para algo que alterna muitas vezes.

v-for: repetição

Sobre um array

```
<script setup>
const characters = ['Gumball', 'Darwin', 'Anais'];
</script>

<template>
  <ul>
    <li v-for="character in characters"
:key="character">
      {{ character }}
    </li>
  </ul>
</template>
```

Sobre objetos, com índice

```
const characters = [
  { id: 1, name: 'Gumball', species: 'Gato' },
  { id: 2, name: 'Darwin', species: 'Peixe' },
];

<p v-for="(character, index) in characters"
:key="character.id">
  {{ index + 1 }}. {{ character.name }}
  ({{ character.species }})
</p>
```

v-for: regras de ouro



:key é obrigatório

Um valor único por item, normalmente o id. Assim o Vue sabe qual elemento é qual ao atualizar a lista.



Também repete números

v-for="pageNumber in 5" gera 1, 2, 3, 4 e 5. Útil em paginação.



Evite v-if e v-for no mesmo elemento

Use um template por fora para o laço e deixe a condição dentro:

```
<template v-for="item in items" :key="item.id">
  <p v-if="item.active">{{ item.name }}</p>
</template>
```

v-model: ligação nos dois sentidos

```
<script setup>
import { ref } from 'vue';

const search = ref('');
</script>

<template>
  <input v-model="search" placeholder="Buscar..." />
  <p>Você digitou: {{ search }}</p>
</template>
```



Digitou no input, a variável muda. Mudou a variável no código, o input acompanha.

É o atalho para :value + @input. Perfeito para formulários e campos de busca.

Criar e usar um componente

Criando components/Greeting.vue

```
<template>
  <p class="greeting">Bem-vindo a Elmore!</p>
</template>

<style scoped>
.greeting {
  font-weight: 700;
}
</style>
```

Usando no Vue puro, com import

```
<script setup>
import Greeting from './components/Greeting.vue';
</script>

<template>
  <Greeting />
  <Greeting />
</template>
```

Escrito duas vezes, aparece duas vezes: é aí que está a reutilização.



No Nuxt, todo arquivo em components/ é importado automaticamente. Nomes em PascalCase (CharacterCard) diferenciam componentes das tags HTML.

Props: passando dados para o componente

Componente CharacterCard.vue

```
<script setup>
const props = defineProps({
  name: String,
  age: Number,
  isMain: Boolean,
});
</script>

<template>
  <article>
    <h3>{{ name }}</h3>
    <p>{{ age }} anos</p>
    <span v-if="isMain">Principal</span>
  </article>
</template>
```

Props são os **parâmetros** do componente: o pai decide o que mostrar, o filho decide como mostrar.

Uso

```
<CharacterCard name="Gumball" :age="12" is-main />
<CharacterCard name="Darwin" :age="10" />
```

Um mesmo componente, dados diferentes, cards diferentes.

Com : e sem :, a parte que mais confunde

Escrita	O que o componente recebe	Tipo
name="Gumball"	o texto "Gumball"	string
:name="characterName"	o valor da variável characterName	o que ela tiver
age="12"	o texto "12" (atenção!)	string
:age="12"	o número 12	number
:is-main="true"	true	boolean
is-main (só o nome)	true, como atalho	boolean
is-main="false"	o texto "false" (atenção!)	string
:tags="['a', 'b']"	um array	array

REGRA DE OURO

Sem : é sempre texto fixo. Com : é JavaScript.

Número, booleano, array, objeto, variável ou expressão precisam de :

Nome da prop

camelCase no defineProps (isMain, fullName) e kebab-case no template (is-main, full-name).

Props são só leitura

O filho não altera uma prop. Se precisar avisar o pai, usa um evento (emit).

Slots: passando HTML para dentro

Prop serve para **dado**. Slot serve para **conteúdo**, um pedaço de HTML.

Slot padrão

```
←!— Card.vue →  
<article class="card">  
  <slot />  
</article>
```

```
<Card>  
  <h3>Gumball</h3>  
  <p>Um gato azul de 12 anos.</p>  
</Card>
```

Tudo entre <Card> e </Card> aparece no lugar do slot.

Slot nomeado, para mais de um espaço

```
←!— FeatureCard.vue →  
<div class="feature">  
  <span><slot name="icon" /></span>  
  <h3>{{ title }}</h3>  
</div>
```

```
<FeatureCard title="245 personagens">  
  <template #icon>  
    <IconsUsers />  
  </template>  
</FeatureCard>
```

Emits: o filho avisando o pai

Filho SearchInput.vue

```
<script setup>
const emit = defineEmits(['search']);
</script>

<template>
  <input
    @input="emit('search', $event.target.value)"
    placeholder="Buscar..."
  />
</template>
```

Pai

```
<SearchInput @search="handleSearch" />
```

Pai

usa o componente



props descem



eventos sobem

Filho

SearchInput

Ciclo de vida: onMounted

```
<script setup>
import { onMounted } from 'vue';

onMounted(() => {
  console.log('O componente apareceu na tela!');
});
</script>
```

Executa um código **quando o componente aparece na tela**.

Útil para tudo que depende do HTML já existir: focar um input, iniciar uma animação, ler o tamanho de um elemento.

No Nuxt, o import também é automático.



Recapitulando a Parte 1

Componente .vue, reatividade (ref, computed), diretivas (v-bind, v-on, v-if, v-for, v-model) e comunicação (props, slots, emits).

02

Nuxt.js

O framework full stack construído sobre o Vue: rotas, servidor e SEO prontos para usar.

NESTA PARTE

- 01 História e vantagens
- 02 Criando o projeto
- 03 Rotas e layout
- 04 Consumindo a API
- 05 Refinamentos e desafio

Como o Nuxt surgiu

O Vue resolvia muito bem a interface, mas cada projeto precisava montar do zero rotas, renderização no servidor e organização de pastas. Inspirados no Next.js do mundo React, os irmãos Alexandre e Sébastien Chopin criaram o Nuxt.

2016

Lançamento

Nuxt nasce para levar renderização no servidor ao Vue com convenções simples.

2018

Nuxt 2

Amadurece e vira referência para sites Vue com SEO.

2022

Nuxt 3

Reescrito com Vue 3, Vite, TypeScript e o servidor Nitro.

2025

Nuxt 4

Nova estrutura com a pasta app/, que usamos no workshop.

2025

Vercel

A NuxtLabs passa a fazer parte da Vercel. O Nuxt segue open source (MIT).

Principais vantagens



Rotas por arquivos

Cada arquivo em pages/ vira uma rota.
Sem configurar router.



Auto-imports

Componentes, composables e funções
do Vue sem escrever import.



SSR e SEO

HTML pronto vindo do servidor, ótimo
para buscadores e redes sociais.



Busca de dados

useFetch busca no servidor e
reaproveita o dado no navegador.



Servidor integrado

Pasta server/ com o Nitro para criar
sua própria API.



Módulos e DevTools

Integrações oficiais e da comunidade
com um comando, além de
DevTools próprio.

Como o Nuxt entrega a página

Vue puro (SPA)

- 1 O navegador recebe um HTML quase vazio
- 2 Baixa e executa o JavaScript
- 3 Busca os dados na API
- 4 Só então a tela aparece

Nuxt (SSR)

- 1 O servidor busca os dados na API
- 2 Monta o HTML completo
- 3 O navegador já mostra o conteúdo
- 4 O Vue hidrata e a página fica interativa



Por isso usamos `await` no `useFetch`: o servidor espera os dados antes de enviar a página, e o navegador não repete a requisição.

Vue e Nuxt lado a lado

Tema	Vue puro	Nuxt
Rotas	Vue Router configurado à mão	Arquivos na pasta pages/
Imports	import em cada arquivo	Automáticos
Renderização	No navegador (SPA)	Servidor, estático ou híbrido
Back-end	Projeto separado	Pasta server/ com Nitro
SEO e head	Bibliotecas extras	useHead nativo
Configuração	vite.config.ts	nuxt.config.ts

Três conceitos antes de começar



Roteamento

Arquivos dentro de `pages/` viram rotas pelo nome. Para rotas dinâmicas, use colchetes:

`pages/[id].vue`



Middlewares

Código que roda antes de entrar numa rota. Três tipos:

Global: todas as páginas

Nomeado: páginas que escolhem usar

Inline: dentro da própria página

`01.auth.global.ts`



Query params

Parâmetros na URL usados para filtrar os dados exibidos ao usuário.

`?filtro=c1813`

Criando o projeto Gumball Verse

Terminal

```
npx nuxi@latest init gumball-verse
```

Pergunta	Resposta
Template	minimal
Package manager	npm
Initialize git repository?	No
Browse and install modules?	No



Template minimal

Começamos com o mínimo possível: só o necessário para entender cada arquivo que entra no projeto.



Sem módulos por enquanto

O estilo vem do nosso próprio design system em CSS, que já está no material do Drive.

Rodar o projeto

```
cd gumball-verse  
npm run dev
```

Prettier: formatação automática

Um **formatador de código**: reescreve os arquivos sempre no mesmo estilo. Ninguém discute vírgula ou indentação, e o código fica igual em qualquer computador.

1. Instalar no projeto

```
npm install prettier --save-dev
```

2. Criar o arquivo .prettierrc na raiz

```
{  
  "trailingComma": "es5",  
  "tabWidth": 4,  
  "semi": true,  
  "singleQuote": true,  
  "printWidth": 100  
}
```

O que cada regra faz

Regra	Valor	Efeito
trailingComma	es5	Vírgula após o último item de objetos e arrays
tabWidth	4	Indentação de 4 espaços
semi	true	Ponto e vírgula no fim das instruções
singleQuote	true	Aspas simples nas strings
printWidth	100	Quebra linhas com mais de 100 caracteres

Prettier no VS Code e no terminal

3. Arquivo `.vscode/settings.json`

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter":
    "esbenp.prettier-vscode",
  "prettier.requireConfig": true,
  "prettier.configPath": ".prettierrc"
}
```

4. `package.json`

```
"scripts": {
  "format": "prettier --write ."
}
```

formatOnSave

Formata a cada Ctrl+S.

defaultFormatter

Usa a extensão Prettier do VS Code (instale antes).

requireConfig

Só formata se o projeto tiver um `.prettierrc`.

configPath

Diz onde estão as regras.

5. Formatar o projeto inteiro

```
npm run format
```

`--write` salva as correções nos arquivos, e o ponto significa: a pasta toda.

A estrutura do projeto

package.json

Dependências e scripts (dev, build, format)

nuxt.config.ts

O coração da configuração do Nuxt

app/

Todo o código da aplicação (Nuxt 4)

app/app.vue

O componente raiz, ponto de partida

public/

Imagens e arquivos servidos como estão

tsconfig.json

Configuração do TypeScript

.gitignore

O que não vai para o Git

.vue e TypeScript

SFCs com lang="ts" quando quiser tipagem

app/app.vue

```
<template>
  <h1>Hello, Nuxt.js! </h1>
</template>
```

Depois de explicar, limpamos arquivos e imagens desnecessários e deixamos o app.vue assim, só para testar.

Primeira requisição com useFetch

```
app/app.vue

<script setup lang="ts">
const { data } = await useFetch(
  'https://gumball-api-server.vercel.app/characters/random'
);
console.log(data.value);
</script>

<template>
  <h1>{{ data[0].name }}</h1>
  <p>{{ data[0].description }}</p>
  <p>{{ data[0].species }}</p>
  <p>{{ data[0].gender }}</p>
</template>
```



useFetch

Composable do Nuxt que faz a requisição e já devolve dados reativos.



data é um Ref

No script, acesse com **data.value**. No template, o ref é desembrulhado.



Por que data[0]?

Essa rota devolve um array com um personagem aleatório.

Rotas a partir de arquivos

pages/index.vue	→ /
pages/characters/index.vue	→ /characters
pages/characters/[id].vue	→ /characters/7
pages/locations/index.vue	→ /locations
pages/locations/[id].vue	→ /locations/3
pages/about/index.vue	→ /about

```
app/app.vue
<template>
  <NuxtLayout>
    <NuxtPage />
  </NuxtLayout>
</template>
```

```
app/error.vue
<template>
  <h1>Página de erro 404</h1>
</template>
```



NuxtPage mostra a página da rota atual, NuxtLayout envolve com o layout, e error.vue aparece sempre que algo der errado ou a rota não existir.

Rotas dinâmicas e useRoute

```
pages/characters/[id].vue

<script setup>
const route = useRoute();
const { id } = route.params;
</script>

<template>
  <h1>Página do personagem de ID {{ id }}</h1>
</template>
```



[id] no nome do arquivo

Os colchetes dizem: esse trecho da URL é variável. /characters/7 e /characters/42 usam a mesma página.



useRoute()

Dá acesso à rota atual. Em route.params está o valor capturado, com o mesmo nome do arquivo: id.

Layout base de todas as páginas

```
app/layouts/default.vue
<template>
  <div class="app-layout">
    <AppHeader />

    <main class="app-layout__main">
      <slot />
    </main>

    <AppFooter />
  </div>
</template>
```

Todas as páginas têm cabeçalho e rodapé. Em vez de repetir em cada página, criamos **um layout**.



O slot de novo!

O `<slot />` é onde cada página aparece. Header e footer ficam fixos, só o miolo muda.

O CSS do layout (flex em coluna e min-height: 100dvh) está no material do Drive.

A base do projeto está no Drive

Para focar no que importa, o Nuxt, a interface base já vem pronta. Baixe e copie para dentro de app/:

- ✓ **assets/css/**
Design system global: cores, fontes, espaçamentos
- ✓ **components/**
AppHeader, AppFooter, cards, filtros, paginação
- ✓ **composables/**
Funções reutilizáveis, como getCharacterColor
- ✓ **public/**
Imagens e favicon do projeto



Link do material

[Acessar a pasta no Google Drive](#)



Personalize

Troque o nome no AppFooter e o GITHUB_USERNAME em pages/about/index.vue pelo seu usuário. A página Sobre puxa seus dados da API do GitHub.

Ajustando o nuxt.config.ts

```
nuxt.config.ts (resumo)

export default defineNuxtConfig({
  css: ['~/assets/css/main.css'],
  components: [
    { path: '~/components/icons', prefix: 'Icons' },
    { path: '~/components', pathPrefix: false },
  ],
  app: {
    head: {
      htmlAttrs: { lang: 'pt-BR' },
      // meta, favicon e Google Fonts
    },
  },
});
```



css

Carrega o design system global em todas as páginas.



components

Ícones ganham o prefixo Icons (IconsUsers) e o resto é usado pelo nome, sem prefixo



app.head

Idioma, descrição, cor do tema, favicon e fontes do site inteiro.

Consumindo a API nas listagens

```
CharactersListing/index.vue

<script setup>
const { data, error } = await useFetch(
  'https://gumball-api-server.vercel.app/characters',
  { query: { limit: 8 } }
);
</script>
```

```
<StateMessage v-if="error">
  Não foi possível carregar os personagens.
</StateMessage>

<div v-else class="grid grid--characters">
  <CharacterCard
    v-for="currentCharacter in data?.data"
    :key="currentCharacter.id"
    :name="currentCharacter.name"
  />
</div>
```

**await**

Garante que os dados chegam antes de a página ser montada no servidor.

**query**

Vira ?limit=8 na URL: a API devolve só 8 itens.

**data?.data**

A resposta tem data (os itens) e meta (paginação). O ?. evita erro enquanto não há dados.

**v-if com error**

Se a API falhar, mostramos uma mensagem amigável.

Páginas de detalhes

```
pages/characters/[id].vue
<script setup>
const route = useRoute();
const { id } = route.params;

const { data } = await useFetch(
  `https://gumball-api-server.vercel.app/characters/${id}`
);
</script>
```

Juntamos a rota dinâmica com o useFetch: o id da URL vira parte do endereço da API.

No template, os dados viram props do CharacterDetails, traduzindo valores da API com ternários:

```
:role="data.role === 'main'
  ? 'Principal' : 'Coadjuvante'"
```

💡 O template completo das páginas de personagem e de lugar está no material do Drive.

Erro 404 e não 500

O problema

Quando o personagem não existe, data fica vazio e a página quebra ao ler data.name. Resultado: um erro 500 genérico, como se o servidor tivesse falhado.

A solução

Tratar o caso logo depois do useFetch e lançar o erro certo com createError. O Nuxt mostra o error.vue com o status correto.

```
pages/characters/[id].vue
const { data, error } = await useFetch(
  `https://gumball-api-server.vercel.app/characters/${id}`
);

if (error.value || !data.value) {
  throw createError({
    status: error.value?.status === 404 ? 404 : 503,
    fatal: true,
  });
}
```

Título das páginas com useHead

```
pages/index.vue

<script setup>
useHead({
  title: 'Gumball Verse',
});
</script>

<template>
  <div>
    <HomeHero />
    <CharactersListing id="characters" />
    <LocationsListing id="locations" />
  </div>
</template>
```

useHead altera o <head> da página: título, meta tags, favicon. Como o Nuxt renderiza no servidor, os buscadores enxergam tudo.

Página	title
/characters	Personagens Gumball Verse
/locations	Lugares Gumball Verse
/about	Sobre Gumball Verse

Filtros e paginação pela URL

```
pages/locations/index.vue
const route = useRoute();

const page = computed(() => Number(route.query.page) || 1);
const type = computed(() => route.query.type);
const search = computed(() => route.query.search);

const { data, error } = await useFetch('https://gumball-api-server.vercel.app/locations', {
  query: { page, type, search, limit: 12, sort: 'id' },
});
```



A URL é a fonte da verdade

?type=school&page=2 fica no link: dá para compartilhar e voltar no navegador.



useFetch reativo

Como page, type e search são computed, mudou a URL, o useFetch busca de novo sozinho.



Chips de filtro

Cada FilterChip é um link: **:to="{ query: { search, type } }"**

Busca com debounce

```
pages/locations/index.vue
let searchTimeout;

function handleSearch(value) {
  clearTimeout(searchTimeout);

  searchTimeout = setTimeout(() => {
    router.replace({ query: { type: type.value, search: value || undefined } });
  }, 400);
}
```



Por que esperar 400 ms?

Sem isso, cada letra digitada faria uma requisição. O debounce espera a pessoa parar de digitar.



router.replace

Atualiza a URL sem criar uma entrada nova no histórico a cada busca.



O emit em ação

O ListToolbar emite search e a página escuta com **@search="handleSearch"**.

DESAFIO

Deixe dinâmicos o título e o ícone das páginas de detalhes

Na página de um personagem ou lugar, a aba do navegador deve mostrar o nome dele e usar a imagem dele como favicon.

Dica 1

Use `useHead` depois do tratamento de erro.

Dica 2

Monte o título com crases (template literal) e `data.value.name`.

Dica 3

As imagens da API são WebP: o tipo do ícone muda.

Resposta do desafio

```
pages/characters/[id].vue e pages/locations/[id].vue
useHead({
  title: `${data.value.name} | Gumball Verse`,
  link: [
    {
      key: 'favicon',
      rel: 'icon',
      type: 'image/webp',
      href: data.value.image,
    },
  ],
});
```



Template literal

As crases permitem encaixar valores no texto: `${data.value.name}`.



key: 'favicon'

A mesma chave do `nuxt.config.ts` faz este ícone substituir o padrão, em vez de duplicar.



image/webp

O favicon padrão é `.ico`. Aqui usamos a imagem da API, que está em WebP.

ENCERRAMENTO

O que construímos hoje

Vue.js

Componentes .vue com script setup e style scoped

Reatividade com ref e computed

Diretivas: v-bind, v-on, v-if, v-for, v-model

Comunicação com props, slots e emits

Nuxt.js

Rotas por arquivos, rotas dinâmicas e layout

Consumo de API com useFetch e query params

Erro 404 correto com createError

SEO com useHead, filtros e busca com debounce



Próximo passo: explore a documentação oficial (vuejs.org e nuxt.com) e evolua o Gumball Verse com novas páginas.

Obrigado!

Perguntas, dúvidas e ideias são muito bem-vindas.



Artur Bomtempo

Chapter Lead no WebTech Network